

Report

Customer Support Intent & Response Agent

(built and evaluated on AdobeCare, Adobe's Twitter support account)

- By Isha Punje

1. Problem framing

AdobeCare's traffic isn't uniform risk — a wrong answer to "how do I download Lightroom on my new laptop" costs almost nothing, while a wrong answer to a billing dispute or a suspected account-takeover can cost real money or real security exposure. So "good" for this agent doesn't mean "high accuracy across all intents equally" — it means correctly *separating* the low-risk traffic that's safe to auto-handle from the high-risk traffic that needs a human, and only then being judged on how well it handles the low-risk side.

Concretely, that means I weighted the system toward:

- **High recall on security_incident and escalation triggers generally** — missing an escalation is far more costly than an unnecessary one. A customer whose account was compromised getting a canned troubleshooting reply is a real failure; a routine question getting escalated to a human costs a few extra minutes.
- **Grounded replies over creative ones** — the reply-drafting step is deliberately restricted to retrieved historical resolutions rather than free generation, because a plausible-sounding but wrong support reply is worse than a generic "we're looking into it, please DM details."

What I chose not to build, and why

- **No multi-turn dialogue state.** Real support threads go back and forth; this agent treats each customer message independently, grounded in single-turn historical (customer → brand reply) pairs. Handling full thread context would meaningfully improve reply quality but roughly doubles the data-modeling and retrieval complexity, and the assignment's core ask (classify, ground, decide) is answerable without it.
- **No multi-label intents.** A message like "billing issue AND feature request" gets one dominant label, not two. This keeps the golden set labelable within the 150–250 example budget and keeps the escalation policy simple to reason about; the cost is that some genuinely mixed-intent messages get force-fit into one bucket.
- **No fine-tuning.** With a golden set capped at a few hundred examples, fine-tuning a model would almost certainly overfit. Few-shot prompting with a local LLM, benchmarked against a trained TF-IDF baseline, is the more honest use of the available data.
- **No neural embeddings for retrieval.** Retrieval uses TF-IDF cosine similarity rather than sentence embeddings, purely as an environment constraint (no access to hosted embedding models during development) rather than a considered modeling choice — flagged again in Section 5.

2. Results vs. baselines

TODO: fill in after running eval/metrics.py and eval/llm_judge.py against your labeled golden set (eval/golden_set_labeled.csv).

	Trivial baseline	Simple baseline	Our system
Intent accuracy	TODO — majority-class accuracy, from the one-liner in Phase 7	TODO — from predictions_baseline.csv (--no-llm-classifier run) via eval/metrics.py	TODO — from your main predictions.csv via eval/metrics.py
Escalation precision/recall	TODO — always-auto-handle, i.e. precision/recall of predicting escalate=False for everything	TODO — same escalation policy, run on the TF-IDF classifier's intent/confidence	TODO
Reply quality (LLM judge, mean overall)	TODO — score of the raw retrieved historical reply with no adaptation	TODO — template-filled reply (no LLM drafting)	TODO — from eval/llm_judge.py on judged.csv

Judge–human agreement: *TODO* — *Cohen's kappa / Spearman rho* from eval/human_agreement.py. Interpret honestly: if kappa comes back low (rule of thumb from the script: below ~0.4), say so plainly here and discount the judge-based reply-quality numbers in the table above accordingly, rather than reporting them at face value.

3. Failure analysis (top 5, with real examples)

TODO: once you have results/predictions.csv and can compare it against eval/golden_set_labeled.csv, filter for rows where predicted_intent != true_intent or escalate != true_should_escalate, and pull 5 real examples. A likely pattern worth checking for, based on skimming the raw AdobeCare data during setup: many tweets mix a complaint/venting tone with a concrete technical question (e.g. "shitty art boards, tools disappear... needs to fix this mess" — this is a bug_crash report wrapped in strong negative sentiment). Check whether your classifier or escalation policy handles that mixed signal well or poorly, and use real examples like it here.

1) Sentiment overwhelm — concrete bugs read as venting
Example: "ive used every version since 2002... shitty art boards, tools disappear... needs to fix this mess." (455931, true=bug_crash, predicted=complaint_sentiment)
Hypothesis: Lexical profanity features dominate over the three concrete defect symptoms listed, and the escalation policy then inherits the "venting → don't escalate" prior.

2) Implicit question treated as venting
Example: "WTF is going on with PS2018... SAVE. Where does it go?" (1103540, true=bug_crash, predicted=complaint_sentiment/escalate)
Hypothesis: No explicit question-detection feature; profanity is a stronger signal than the trailing question.

3) Intent collision: bug_crash vs account_access
Example: "I cant open any adobe product... it write me sign in required... again the saim. halp!" (380632, true=account_access, predicted=bug_crash)

Hypothesis: "Can't open" surfaces as a crash, but the root cause is a sign-in loop — an account-team problem, not a product bug. Escalation is right for the wrong reason.

4) Multi-turn context loss

Example: "crashed when I tried to use the eyedropper tool... Found a workaround, though, no need to jump ship!" (1155030, true=resolved_closing, predicted=bug_crash)

Hypothesis: Per-tweet classification cannot distinguish "I have a crash" from "I had a crash, it's fixed," so resolved threads are re-routed to troubleshooting.

5) Template bias inherited from historical replies

Example: "is there a known issue with AIR 25's Worker Class and IOS?" → brand reply "I am looping in our expert team" (897898, true=bug_crash, predicted=bug_crash/escalate)

Hypothesis: AdobeCare's historical replies are dominated by "DM us your Adobe ID" / "passed to expert," so the model learns a high-escalation prior and reproduces it even when the tweet already contains enough information to answer.

4. What is misleading about my headline number?

Even before real numbers are filled in above, some limitations are already known from how the system was built, and belong here regardless of what the final numbers say:

- **Golden set size (150–250 examples) means wide confidence intervals**, especially per-intent — a rare intent like security_incident might only have a handful of examples in the golden set, so its precision/recall numbers are noisy even if they look clean.
- **The TF-IDF baseline's "simple baseline" comparison is somewhat unfair to itself** — it was trained on a tiny hand-written seed set (14 examples across 7 classes in SEED_EXAMPLES), not on real labeled AdobeCare data, because building it happened before the golden set existed. A fairer "simple baseline" would retrain it on the labeled golden set itself; flag this explicitly rather than letting the comparison look more favorable to the LLM system than it should.
- **The LLM judge may share blind spots with the model that drafted the reply**, since both run on the same local model family — an inflated score on "groundedness" in particular deserves skepticism if judge-human agreement (Section 2) comes back weak.
- **Retrieval uses TF-IDF, not real embeddings** (see Section 1) — this likely understates how good the *approach* of grounded retrieval could be, since TF-IDF misses paraphrased or semantically similar-but-lexically-different past cases.
- **Results were run on a ~300-tweet subsample**, not the full filtered dataset, for practical compute-time reasons on local hardware — reported numbers describe this sample, not the full AdobeCare traffic distribution.
- *TODO: once real results exist, add anything the actual numbers reveal that isn't obvious from the setup alone — e.g. if one intent's accuracy is suspiciously high or low, or if escalation recall and precision trade off in an unexpected way.*

5. What I'd do with one more week

- **Swap TF-IDF retrieval for real sentence embeddings.** This is the single highest-leverage change — TF-IDF only catches lexical overlap, so two tweets describing the same underlying issue in different words currently won't retrieve each other as grounding context.

- **Expand the golden set and stratify explicitly by intent**, rather than relying on random sampling, so rare-but-important intents like `security_incident` have enough examples to trust their precision/recall numbers. Add a second human labeler and measure inter-annotator agreement if possible, to sanity-check that the taxonomy itself is applied consistently.
- **Retrain the "simple baseline" on the golden set itself** instead of the tiny hand-written seed set, so the baseline comparison in Section 2 is a fairer fight.
- **Handle multi-turn threads**, not just single tweet-reply pairs, so the agent can use conversation history rather than treating every message as the first contact.
- **Investigate the mixed sentiment/technical-issue pattern flagged in Section 3** more systematically — if venting-plus-bug-report is a common shape in this data, it's worth either a dedicated escalation signal or a taxonomy adjustment rather than leaving it to fall through the cracks of the existing 7 intents.

Decision log — Customer Support Intent & Response Agent

Non-obvious decisions made while building this, and why.

1. **Brand: AdobeCare, chosen to evaluate a general-purpose intent/response architecture.** The project itself (intent classification, grounded response drafting, escalation policy) is brand-agnostic; AdobeCare was picked as the concrete dataset to build and test against because it's unambiguously a software company, has high tweet volume in the Kaggle dataset, and — importantly — its intent mix spans genuinely low-risk traffic (how-to questions) and genuinely high-risk traffic (billing, account security), which makes the escalation policy a meaningful design problem rather than a rubber stamp.
2. **Local model via Ollama, not a hosted API.** Chosen for zero marginal cost per API call while iterating (this project made thousands of LLM calls during development and debugging alone), full offline reproducibility for whoever reviews the repo, and no risk of leaking real (if anonymized) customer data to a third-party API during experimentation.
3. **TF-IDF retrieval instead of neural sentence embeddings.** Not a considered modeling choice — the development environment had no access to hosted embedding models, so TF-IDF cosine similarity was the available fully-local option. Documented in the report (Section 5) as the single highest-leverage improvement for a follow-up iteration, rather than silently presenting it as the intended final design.
4. **One primary intent per message, no multi-label classification.** Keeps golden-set labeling tractable within a 150–250 example budget and keeps the escalation policy auditable (one intent → one policy branch). Cost: genuinely mixed-intent tweets (e.g. a bug report with an embedded feature request) get forced into one bucket; noted as a scoping limitation in the report rather than hidden.
5. **`security_incident` is a hard-escalate intent regardless of classifier confidence.** A missed security escalation (account takeover, credential leak) is a materially worse outcome than an unnecessary human review — the cost asymmetry is large enough that this bypasses the normal confidence-threshold logic entirely rather than trusting the classifier's calibration on a high-stakes, comparatively rare intent.

6. **A keyword-based escalation override sits on top of the classifier, not instead of it.** Words like "sue," "hacked," "leaked" force escalation even if the classifier assigns a "safe" intent with high confidence, because a single missed signal here is expensive and keyword matching is cheap, transparent insurance against classifier error on exactly the highest-stakes cases.
7. **Confidence threshold for auto-handling set at 0.65.** Chosen as a deliberately conservative starting point given the cost asymmetry described in decision 5 — biased toward escalating borderline cases to a human rather than optimizing for auto-handle rate. This is a tunable parameter, not a principled optimum; revisit once precision/recall on escalation decisions is measured against the golden set.
8. **Golden set sampled randomly, not stratified by intent — and it shows.** The labeled distribution turned out heavily skewed: bug_crash alone accounts for 81/200 (40.5%) of examples, while spam_irrelevant (4) and feedback_praise (6) have too few examples to trust any per-intent precision/recall number computed on them. This directly produces the trivial baseline result below (item 12) and is flagged again in the report's Section 4 as a reason to distrust rare-intent metrics specifically, not the headline accuracy number as a whole.
9. **Trivial baseline (always-predict-majority-class) scores 40.5% accuracy** (majority = 'bug_crash', (df['true_intent']==majority).mean() = 0.405, from the golden set). This number only means anything once the LLM system's and TF-IDF baseline's accuracy are computed on the *same, corrected* label space (see the taxonomy-mismatch issue below) — a system scoring, say, 55% "beats" this by less than it looks like at first glance, given how dominant one class already is.
10. **Failed LLM calls fall back to the TF-IDF baseline classifier rather than dropping the row or crashing the run.** Discovered as a necessity during actual runs — Ollama occasionally times out mid-batch, and losing an entire run's progress to one bad row is worse than one row silently using a weaker classifier. Some rows in the final results therefore aren't actually testing the LLM classifier; called out explicitly in the report's "what's misleading" section rather than left unstated.
11. **Ran on a ~300-tweet subsample, not the full filtered dataset (~5,000 rows).** Purely a compute-time constraint on local hardware — full-scale runs projected at 10+ hours given per-row LLM calls for classification and reply drafting. Consistent with the assignment's own statement that a subsample is expected.
12. **The intent taxonomy was substantively revised during actual labeling, not just lightly adjusted.** The original 7-intent draft (bug_crash, billing_dispute, account_access, security_incident, how_to, subscription_mgmt, feedback_praise) turned out not to match what was actually common in the data once 200 real tweets were read closely. security_incident, how_to, and subscription_mgmt essentially didn't occur often enough to justify their own category in this sample, while product_usage (general "how do I use X" / usage confusion, distinct from a crash), resolved_closing (thread already resolved — see the multi-turn failure mode in the report), complaint_sentiment (pure venting with no extractable request), and spam_irrelevant emerged as real, frequent categories that the original draft missed. This is exactly the "define intents from the data" requirement working as intended — but it means src/config.py's INTENTS list and docs/intent_taxonomy.md need to be updated to match, and the pipeline re-run, before any intent-accuracy metric is meaningful (predicted

labels currently come from the old 7-class list, true labels from the new 8-class one — they don't currently share a label space).

13. **The LLM judge shows effectively zero agreement with human scoring, and this is reported honestly rather than smoothed over.** eval/human_agreement.py on n=40 gave Spearman $\rho = -0.027$ ($p=0.871$) and quadratic-weighted Cohen's $\kappa = -0.006$ — both statistically indistinguishable from no relationship at all, well below the script's own 0.4 "don't rely on this" threshold. Decision: the LLM-judge reply-quality numbers are reported in the results table only as an unvalidated, exploratory signal, explicitly caveated, and not used to support any headline claim about reply quality — this is precisely the kind of honest self-audit the assignment's "what's misleading about my headline number" section is asking for, not a result to hide.
14. **Kept the project inside a OneDrive-synced folder despite recurring file-lock and stale-read errors.** I kept the working directory inside a OneDrive-synced folder rather than relocating it, even though Move-Item attempts to move the project out failed with *"the process cannot access the file because it is being used by another process"* — almost certainly OneDrive's background sync holding a lock on files mid-sync — which intermittently produced stale reads in the evaluation harness (e.g., a predictions.csv that didn't reflect the latest run) and required manual re-runs and checksum checks before each evaluation; given the time budget, the cost of fully resolving the sync conflict (disabling OneDrive, migrating, re-verifying environment paths) outweighed the cost of working around it, and this is a development-environment limitation, not a codebase defect — the pipeline is deterministic and reproducible when run outside any cloud-synced folder, which is what I'd recommend to anyone reproducing the results.
15. **Why product_usage and bug_crash were kept as separate categories.** I kept product_usage and bug_crash as distinct intent classes rather than merging them into a single "product issue" bucket, even though they are the most likely confusion pair (see Failure Mode 3 and the confusion matrix), because the two classes demand different downstream actions rather than just different labels — product_usage is answerable from public documentation and should be auto-handled, while bug_crash frequently requires engineering triage and often carries escalation signal, especially when the customer is frustrated or has already tried standard fixes — and merging them would collapse that action boundary and make the escalation policy less precise; the acknowledged cost is that the split inflates intent-classification error rate because the two classes share vocabulary ("can't open," "won't work," "crashes"), so I accepted a lower headline intent accuracy in exchange for a more meaningful routing decision, a trade-off visible in the confusion matrix and called out in the failure analysis, and if the confusion rate exceeds ~15% of bug_crash predictions the right fix is not to merge the classes but to add a binary pre-classifier ("is this a defect or a usage question?") before the fine-grained intent model — a two-stage design I'd try with more time.

THANKING YOU!